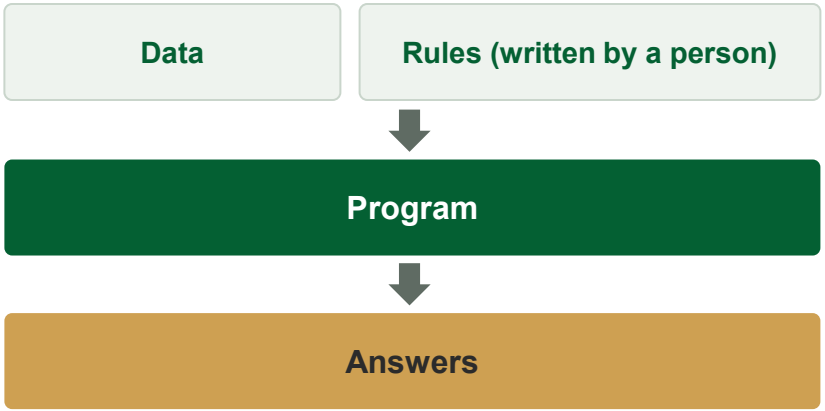
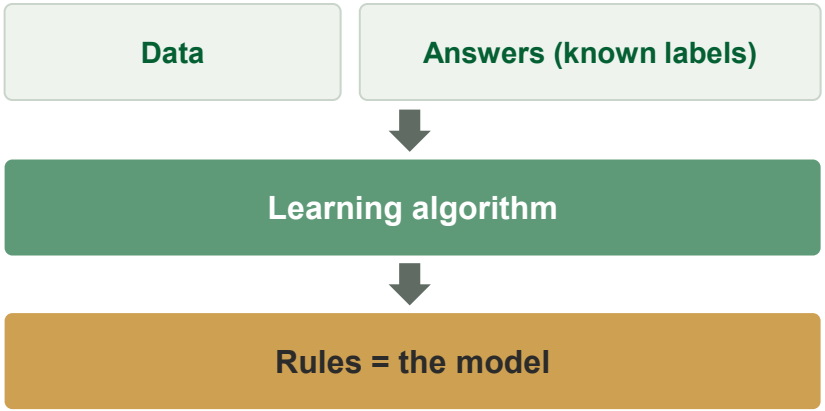
An introduction to modelling

Traditional programming vs predictive modelling

Traditional programming



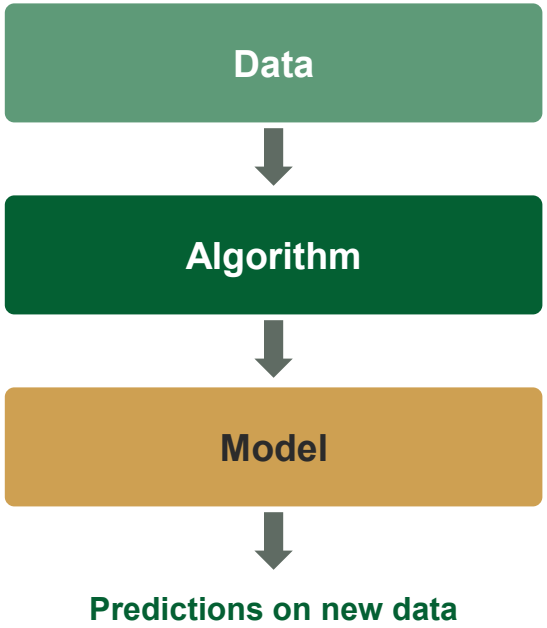
Predictive modelling



Spam example: a programmer writes "if the email contains FREE, mark spam" and keeps patching it. A model instead learns from thousands of labelled emails which word patterns signal spam, then applies that rule to new mail. In predictive modelling the output of training is the rules (the model), and the model is then used to predict.

What is a machine learning model?

- **Model =** a mathematical function that maps inputs (features) to an output (target): y = f(x).
- **Learned, not hand-coded:** an algorithm adjusts the function using example data.
- **Example:** predict house price from area, bedrooms and location.
- **Training** finds the best parameters. Prediction applies them to new houses.



When is machine learning the right tool?

✓ ML is a good fit when

- The rule is hard to write down (spam, faces, speech).
- Patterns change over time and rules would need constant patching.
- You have many examples, ideally with known outcomes.
- Some errors are acceptable and can be measured.

✗ Prefer rules or simple code when

- A known formula or rule already exists (payroll, tax, unit conversion).
- There is little or no relevant data.
- Every answer must be exactly right and explainable by law.
- A simple heuristic already does the job well enough.

Problem framing (Google): first decide if ML is the right approach, then frame it in ML terms. Always write down a non-ML baseline the model must beat.

Algorithm, model, parameters, hyperparameters

1 Algorithm

The procedure or set of steps used to learn patterns or solve a problem.

Example: linear regression, random forest.

2 Model

The result after training: The algorithm plus its learned numbers.

3 Parameters

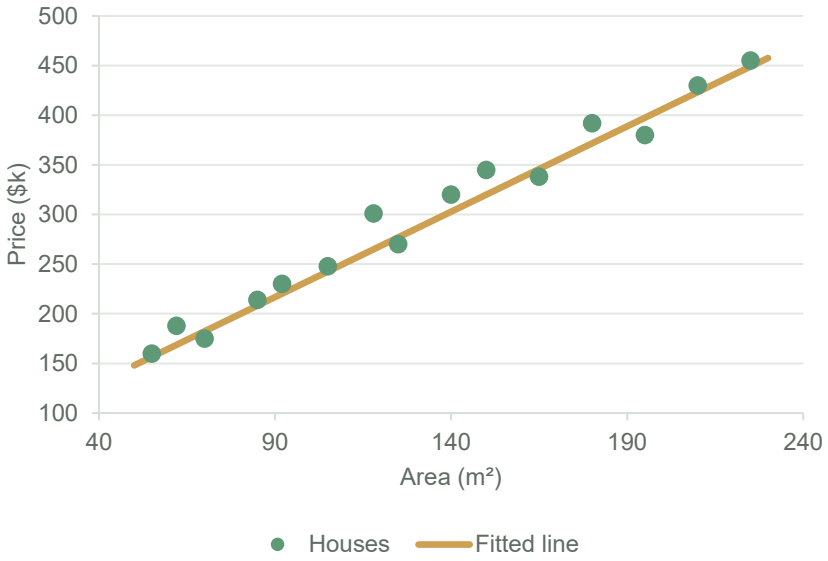
Learned from data. Example: coefficients, tree split values. You do not set them.

4 Hyperparameters

Set by you before training. Example: k in kNN, max_depth, learning_rate.

Rule of thumb: parameters come from training data, hyperparameters are tuned using validation data. Tuning on the test set is cheating.

How does a model learn? Model family, loss, optimise



1 Pick a model family

The set of functions allowed. Here: all straight lines, price = w × area + b.

2 Define a loss

A score for how wrong a prediction is. Here: squared gap between dot and line.

3 Optimise

Find the w and b with the lowest average loss on the training data.

Deep learning uses the same recipe: the model family is a large, layered neural network that learns its own features from raw pixels, words or sound.

Six ways a machine can learn

1 Supervised

Every example comes with the correct answer. Predict spam, price, diagnosis.

2 Unsupervised

No answers. Find structure: customer segments, topics, unusual records.

3 Semi-supervised

A few labelled examples plus many unlabelled ones. Useful when labelling is costly.

4 Self-supervised

The data labels itself: hide part of the input and predict it. Powers today's large models.

5 Reinforcement

An agent acts, receives rewards or penalties, and learns a strategy. Games, robots, LLM tuning.

6 Transfer learning

Start from a model already trained on huge data, then adapt it to your task with little data.

Supervised learning: learn from answered examples

Learn from examples that come with the correct answer, then predict the answer for new cases.

Input (x)	Answer (y)
Email text	Spam / not spam
Chest X-ray	Pneumonia: yes / no
House details	$214k
New email	? model predicts

Use it when

- You have historical examples with known outcomes
- The same kind of question will be asked again

Real examples

- Spam filtering; loan default risk
- Reading X-rays; estimating house prices

Watch out: the model copies whatever is in the labels, including human errors and biases. Check label quality first.

Unsupervised learning: find structure without answers

No answer column: the model finds groups, patterns or unusual records in the inputs alone.

Customer	Visits / mo	Spend	Group
A	2	$15	?
B	14	$220	?
C	3	$12	?
D	12	$240	?

The model might group A with C (occasional, low spend) and B with D (frequent, high spend). People decide what the groups mean.

Use it when

- No labels exist, or you want to explore first
- You need groups, a summary, or outliers

Real examples

- Customer segments; topics in documents
- Flagging unusual transactions or sensor readings

Watch out: with no answer key, results are hard to check. A person must judge whether the structure is real and useful.

Semi-supervised learning

Combine a small labelled set with a large unlabelled set to learn more than the labels alone allow.

Scan	Label
scan_001	Tumour
scan_002	*not labelled*
scan_003	*not labelled*
scan_004	Healthy
... 10,000 more	*not labelled*

Use it when

- Labels need experts and are expensive
- Unlabelled data from the same source is plentiful

Real examples

- Medical scans with few annotations
- Speech with only some recordings transcribed

Watch out: confident wrong guesses get fed back in as labels and can reinforce mistakes. Keep a clean labelled test set.

Self-supervised learning

Text	The cat sat on the ____	**Predict the hidden or next word**
Images	Hide 75% of an image's patches	**Reconstruct the missing patches**
Audio	Mask a stretch of speech	**Predict what was masked**

Why it matters

- Unlabelled data is cheap and plentiful.
- Learns general features that transfer to many tasks.
- Pretraining step behind GPT-style LLMs and modern vision and speech models.

Not the same as unsupervised: there is a target, as in supervised learning, but it is cut out of the input itself instead of being supplied by a person.

Reinforcement learning: trial, reward, penalty

No answer key: an agent acts, gets rewards or penalties, and learns which actions pay off over time.



The loop repeats thousands or millions of times; the agent keeps the actions that earn more reward.

Use it when

- Decisions come in a sequence and affect what happens next
- You can measure outcomes and simulate safely

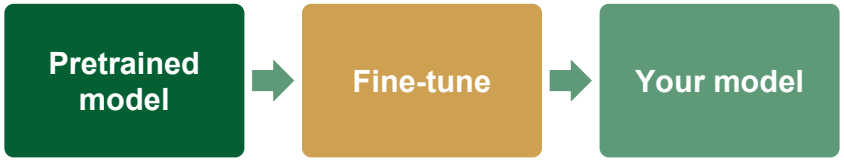
Real examples

- Game playing; robot walking and grasping
- Refining chatbot answers from feedback

Watch out: a poorly designed reward gets gamed, and real-world trial and error can be slow, costly or unsafe.

Transfer learning: reuse a pretrained model

Reuse what a pretrained model learned from huge data, then adapt it to your task with a small labelled set.



Today this is usually the first thing to try for text, images and audio.

Use it when

- Your labelled dataset is small
- A pretrained model exists for your type of data

Real examples

- Adapt a language model to clinical notes
- Detect crop disease from 500 leaf photos

Watch out: the pretrained data may differ from yours, or may already include your test data. Test on truly new data.

How today's AI models are built?

1 Pretrain

Self-supervised

Predict the next word on trillions of words of unlabelled text.

2 Fine-tune

Supervised

Learn from prompts paired with ideal answers written by people.

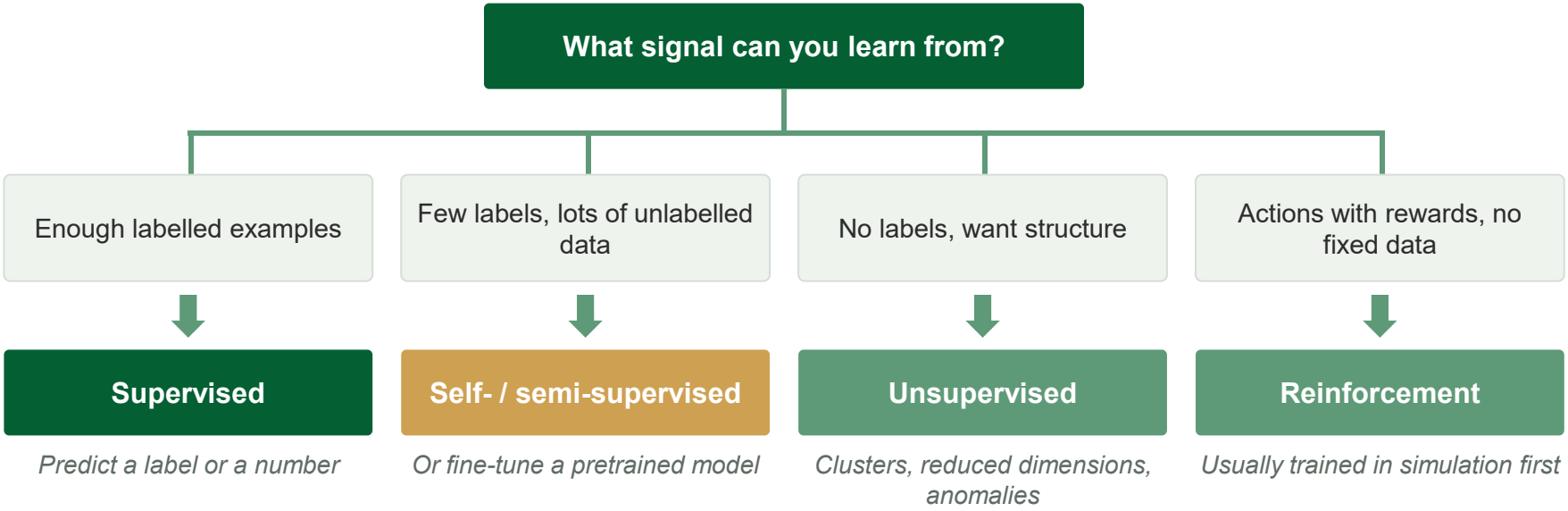
3 Align and refine

Reinforcement

Rewarded for answers people, other models or checkers rate higher.

What this means for your projects: you rarely train a large model from scratch. Start from a pretrained model and adapt it with your own labelled data (transfer learning). The splitting and leakage rules later in this lecture still apply in full.

When to use what: a decision guide



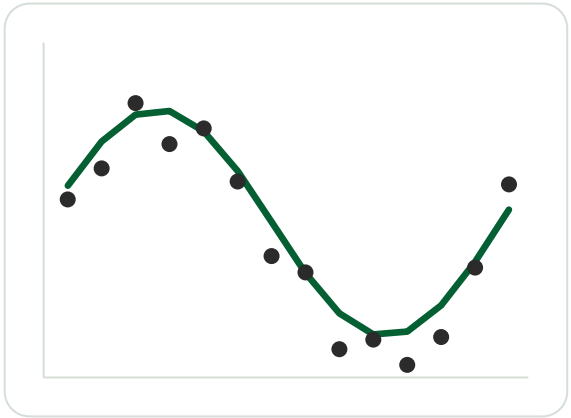
Every branch ends the same way: start with a simple baseline and add complexity only if it wins on data the model has not seen.

Underfitting vs overfitting



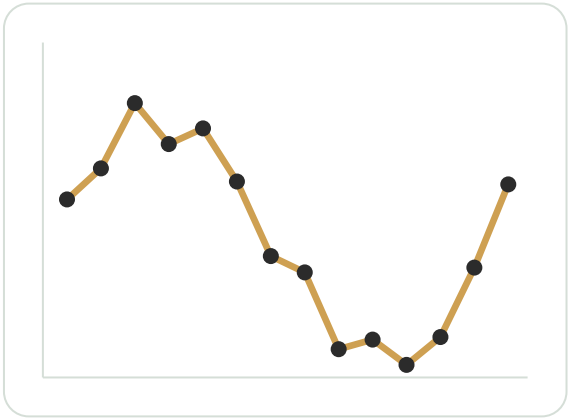
Underfitting (High bias)

Train error: high | Test error: high



Good fit (Balanced)

Train error: low | Test error: low



Overfitting (High variance)

Train error: ~0 | Test error: high

The tell-tale sign: overfitting shows as low training error but much higher validation error. Underfitting is poor on both.

Hold-out split: train, validation and test



- **Train:** The model learns its parameters here.
- **Validation:** Compare models and tune settings, as often as you like.
- **Test:** Locked away and used once, at the very end.

Rules of thumb

- Typical ratios: 70/15/15 or 80/10/10.
- Stratify: keep the same mix of outcomes in every part.
- Fix a random seed so the split is reproducible.
- Split first, then do any scaling or cleaning on the training part only.

Weakness: one split is one roll of the dice. With small data the score depends on which rows land in the test set, so use cross-validation.

k-fold cross-validation: every row gets tested once

Gold = validation fold in that round

R1	Val	Train	Train	Train	Train
R2	Train	Val	Train	Train	Train
R3	Train	Train	Val	Train	Train
R4	Train	Train	Train	Val	Train
R5	Train	Train	Train	Train	Val

1 Cut

Shuffle the rows and cut them into k equal folds (k = 5 or 10).

2 Rotate

Train on k-1 folds, validate on the fold left out. Repeat k times.

3 Report

Give the mean and the spread of the k scores, not just the best one.

Variants: stratified k-fold keeps the outcome mix in every fold; leave-one-out sets k = number of rows (only for tiny datasets).

Leave-one-subject-out: when rows are not independent

	S1	S2	S3	S4	S5
Fold 1	Test	Train	Train	Train	Train
Fold 2	Train	Test	Train	Train	Train
Fold 3	Train	Train	Test	Train	Train
Fold 4	Train	Train	Train	Test	Train
Fold 5	Train	Train	Train	Train	Test

Each column = all rows of one person. Each person is tested once.

Why a random split fails

- One person gives many similar rows.
- Random split puts them in train and test.
- The model recognises the person, so the score is inflated.

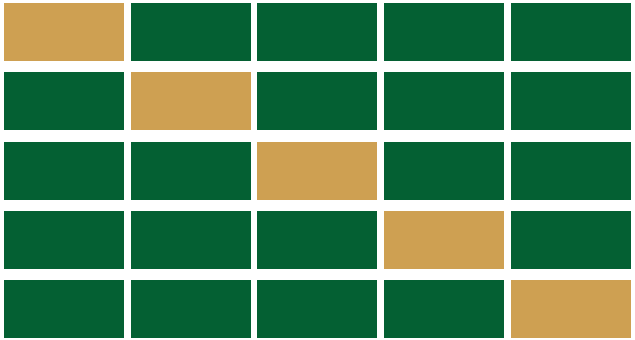
Related splits

- Group k-fold: hold out a group of subjects per fold.
- Leave-one-site-out: hold out a whole hospital or device.
- Report mean and spread across subjects.

Rule: if the model will be used on new people, test it on people it has never seen.

Nested cross-validation: tune and test without cheating

Outer loop: 5 folds (honest score)



Gold = outer test fold

Inside each green outer training set



Inner loop: 3-fold CV tries each hyperparameter setting and picks the best.

Then: refit on the whole outer training set and score once on the gold fold.

Why: tuning and reporting on the same folds inflates the score. The inner loop tunes, the outer loop measures. Report the mean and spread of the five outer scores.

Choosing a split: mimic how the model will be used

Strategy	How it works	Use when	Watch out
Hold-out	One train / validation / test split	Large dataset	One lucky or unlucky split
Stratified hold-out	Same outcome mix in each part	One outcome is rare	Still only one split
k-fold	k parts, each validated once	Small or medium data	k times the training cost
Stratified k-fold	k-fold keeping the outcome mix	Rare outcomes, small data	Rows must be independent
Leave-one-out	k = number of rows	Very small dataset	Slow; noisy estimate
Group k-fold / LOSO	All rows of a subject stay together	Many rows per person or device	Fewer, uneven folds

Data leakage: when test information sneaks in

- **Data leakage:** information from outside the training data (the test set or the answer itself) sneaks into model building.
- **Result:** a great score in the notebook, poor results in real use.
- **Why a separate test set:** it stands in for future data. The moment test data influences any choice (features, scaling, tuning), it is no longer unseen and the estimate turns optimistic.

Type	Example	Fix
Target leakage	Feature "antibiotic prescribed" used to predict infection	Drop features not known at prediction time
Preprocessing leakage	Scaling or imputing on all data before splitting	Fit on train only, then apply to test
Duplicate / same person	One patient has rows in both train and test	Split by person (group), not by row

Red flag: near-perfect accuracy on your first try. Check for leakage before celebrating.

Leakage in the foundation-model era: contamination

! What goes wrong

- Pretrained models learn from huge web scrapes.
- Public benchmarks and datasets are often on the web too.
- The model may have seen your test questions, so a high score reflects memory, not skill.

✓ How to protect your evaluation

- Keep a private test set that has never been online.
- Use data collected after the model's training cut-off.
- Check for duplicates between test data and fine-tuning data.
- Treat public leaderboard scores with caution.

Same rule as always: the test set must be data the model has truly never seen. With pretrained models you have to check, not assume.